DISEÑO AVANZADO DE BASES DE DATOS

EL MODELO DE DATOS RELACIONAL (Recordatorio)

Organiza y representa los datos en tablas o relaciones. Una tabla o relación describe una asociación entre elementos de dos o más conjuntos. Formalmente, una relación entre dos conjuntos A y B (entidades) es un subconjunto del producto cartesiano AxB , que contiene pares ordenados (a, b) donde $a \in A$ y $b \in B$

Conceptos:

- Atributo (A_i): Cualquier elemento de información susceptible de tomar valores.
- Dominio (D_i): Rango de valores que puede tomar un atributo. Se considera finito
- Esquema de una relación R: Atributos de la relación junto con su dominio. $A_1: D_1,... A_n: D_n$
- Relación $R(A_1,..., A_n)$ Dados los atributos $A_1,..., A_n$ con sus respectivos dominios, definimos relación asociada ellas como:
  $R = \{A_1: D_1,\ A_2: D_2,..., A_n: D_n\}$; instancia de la tabla: $r \in D_1 \times ... \times D_n$
- Tupla $t_i \in r$: cada una de las filas de una relación (elementos del subconjunto $D_1 \times ... \times D_n$)
- Cardinalidad de una relación: número de tuplas que contiene. Es variable en el tiempo.
- Grado de una relación: Número de atributos de su esquema. Invariable en el tiempo
- Instancia de una relación $R: r$: Conjunto de tuplas $\{(x_1, x_2,..., x_n) \in D_1 \times ... \times D_n\}$ que la componen en cada momento
- Esquema de una base de datos relacional: Colección de esquemas de relaciones junto con restricciones de integridad que definen las configuraciones válidas de los datos
- Instancia de una base de datos: Colección de instancias de relaciones que verifican las restricciones de integridad. Válida si respeta las reglas de integridad.

Restricciones de integridad

Las restricciones o reglas de integridad son condiciones para preservar la corrección semántica de una base de datos.

- Superclave : Cualquier conjunto de atributos que identifica unívocamente a cada tupla de una relación. (Sólo cumple unicidad y no minimalidad)
- Clave Candidata CC: superclave minimal:

Sea $R(A_1,..., A_n)$ una relación; un conjunto de atributos CC $\subseteq \{A_1,..., A_n\}$ se denomina clave candidata si y sólo si:

Unicidad: $\forall r \in R$ instancia de relación y cualesquiera dos tuplas diferentes $t_1,\ t_2 \in r$ tales que $t_1 \neq t_2 \Rightarrow t_1[CC] \neq t_2[CC]$

Minimalidad: No existe CC'⊂CC que verifique la unicidad (conformada por los mínimos atributos)

Los atributos que conforman una clave candidata se denominan <u>atributo primo</u>.

- Clave primaria CP: Es la clave candidata elegida por el diseñador para desempeñar el papel de identificar (sólo una por relación)
- Clave externa CE: Si CE es una clave externa de R que referencia a CP en S, entonces: $\forall t \in r,\ \exists t' \in s$ tal que $t[CE] = s[CP]$. Eventualmente, R y S pueden ser la misma relación (puede haber una clave externa a la clave primaria de la propia relación). Puede haber más de una clave externa en una relación.

<u>Reglas de Integridad Genérica:</u> Son metarreglas (generan reglas de integridad aplicadas a una base de datos concreta). Depende del papel que juega un atributo en el diseño de la tabla.

- Integridad de entidad: Un atributo que forma parte de la clave primaria de una tupla en una relación no puede tener un valor nulo.
- Integridad referencial: Si una relación incluye una clave externa conectada a una clave primaria, el valor de la clave externa debe ser, bien igual a un valor ya existente en el dominio activo de la clave primaria, o bien completamente nulo (si la semántica lo permite).

Restricciones específicas: aquellas que provienen de la semántica del atributo y son propias de cada base de datos concreta. Son específicas del problema (Ej: o<edad<100) y propias del papel que desempeña cada atributo en el esquema.

DISEÑO LÓGICO RELACIONAL: Normalización

Partimos de un esquema conceptual de nuestro sistema (esquema entidad-relación) para generar un esquema relacional (estructura de tablas lógicas). Este proceso se conoce como paso a tablas o Diseño Lógico Relacional.

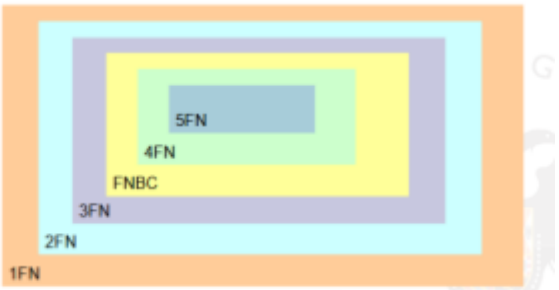
A la hora de hacerlo, refinamos mediante la Normalización para:

- Corregir defectos del modelo conceptual
- Eliminar redundancias, problemas de actualización (en inserción, borrado, modificación)
- Plasmar restricciones semánticas adicionales (que las tablas digan **exactamente lo que quieren decir** y no cosas ambiguas).

El "buen comportamiento" de una relación se denominan formas normales. Son como **sellos de calidad**. Si una tabla cumple una forma normal, significa que hemos eliminado una clase de mal diseño.

Forma Normal	Año	Autor	Basada en...
1FN	1970	Codd	No basada en dependencias: Impone dominios atómicos
2FN	1970	Codd	Dependencias funcionales
3FN	1970	Codd	
FNBC	1972	Boyce & Codd	
4FN	1977	Fagin	Dependencias multivaluadas
5FN	1979	Rissanen	Dependencias de reunión

Relación entre conjuntos de instancias que cumplen cada una de las formas normales



Primera forma normal (1FN)

Decimos que una relación está en Primera Forma Normal, cuando todos los valores de los atributos de esa relación son atómicos (no estructurados). De esta manera no hay valores tipo conjunto, ni tipo registro, ni tipo tabla.

El problema es que todas las representaciones son extensivas, no es posible almacenar directamente múltiples valores para un solo atributo sin crear una entrada separada para cada uno de ellos.

Definición: Dependencia funcional: Dada una relación R con $n > 0$ atributos y dos subconjuntos de atributos de R llamados a y b, se dice que a determina funcionalmente a b ($\equiv$ b depende funcionalmente de a), notado como $a \rightarrow b \Leftrightarrow$ para cualquier instancia r de R y $\forall t, s \in r$ tuplas $t[a] = s[a] \Rightarrow t[b] = s[b]$
(la combinación de valores de a determina la combinación de valores de b)

Las dependencias funcionales establecen restricciones semánticas conocidas y que deben verificarse en los datos. Por ejemplo; $DNI \rightarrow Nombre$

- Dependencia Funcional Completa: En el contexto descrito, a determina funcionalmente a b de forma completa (notamos $a \rightarrow\rightarrow b$) $\Leftrightarrow a \rightarrow b \ \wedge \ \nexists c \subset a$ tq $c \rightarrow b$.
  Es decir, a está compuesto por la mínima cantidad de atributos que determina funcionalmente a b)
- Dependencia trivial: toda dependencia $a \rightarrow b$ tal que $b \subseteq a$.

Axiomas de Armstrong:

- Reflexividad: $\forall\, a, b \mid b \subseteq a$ se verifica $a \to b$
- Ampliación: $\forall\, a, b, c \mid a \to b$ se verifica $ac \to bc$
- Transitividad: $\forall\, a, b, c \mid a \to b \wedge b \to c$ se verifica $a \to c$

Reglas de Armstrong:

- Unión: $\forall\, a, b, c \mid a \to b \wedge a \to c$ se verifica $a \to bc$
- Descomposición: $\forall\, a, b, c \mid a \to bc$ se verifica $a \to b \wedge a \to c$
- Pseudotransitividad: $\forall\, a, b, c, d \mid a \to b \wedge bc \to d$ se verifica $ac \to d$

Segunda forma normal (2FN)

Una relación R está en 2FN si y sólo si:

I. Está en 1FN y
II. Todos sus atributos no primos dependen de forma completa de las claves candidatas.

Tener I. es sencillo, pero II. no lo es tanto. Puede ocurrir que en ciertas relaciones, algunos de los atributos no primos dependen de forma no completa de la clave candidata, pues están determinadas por parte de ella. En ese caso, podemos usar:

<u>Teorema de Heath:</u>

Sea (R, r) una relación y $A,\ B,\ C$ tres conjuntos de atributos que forman una partición del esquema R, y donde se verifica la dependencia no trivial $B \to C \Rightarrow$ la descomposición de R en dos relaciones:

- $R_1(A, B)$
- $R_2(B, C)$

es una descomposición sin pérdidas (esto quiere decir que $R_1 \cup R_2 = R$, y $\forall r_1 \in R_1,\ r_2 \in R_2,\ r_1 JOIN\, r_2 = r$. Podemos recuperar R mediante reunión natural ≡Alternativamente, la descomposición es sin pérdidas si la intersección de los esquemas de ambas tablas determinan funcionalmente al esquema completo de una de ellas).

<u>Preservación de dependencias en descomposiciones sin pérdidas</u>

Al descomponer puede parecer haber perdido dependencias. Para verificar si las dependencias siguen existiendo sin que estén explícitamente presente se emplea:

- Cierre de un conjunto de dependencias funcionales Φ (Φ^+): Todas las dependencias funcionales que pueden deducirse de las que tenemos Φ aplicando los Axiomas y las Reglas de Armstrong.
- Cierre de un conjunto de atributos a dado un conjunto de dependencias funcionales Φ (a^+_Φ): representa qué atributos puedes conocer si te dan inicialmente un conjunto a y aplicas dependencias de Φ^+. Ambos cierres son equivalentes ya que: $A \in a^+_\Phi \Leftrightarrow a \to A \in \Phi^+$
- Cálculo del cierre de un conjunto de atributos de a en base a conjunto de dependencias funcionales Φ:
  - Inicializamos $a^+_\Phi = a$
  - Mientras que a^+_Φ cambie: Si $b \to c \in \Phi$ y $b \subseteq a^+_\Phi \Rightarrow a^+_\Phi = a^+_\Phi \cup c$ (por el axioma de transitividad).

Tercera forma normal (3FN)

Una relación R está en 3FN ⇔ para toda dependencia $a \to b$ no trivial se cumple:

I. a es una superclave. ó
II. b contiene sólamente atributos primos (parte de alguna clave)

Esta definición equivale a que una relación R con F un conjunto de dependencias funcionales (DFs) asociado, está en tercera forma normal (3FN) sii

- Está en 2FN, y
- No presenta dependencias transitivas problemáticas (DTP: $CC \rightarrow b$ es una DTP, con b atributo no primo si $CC \rightarrow b$ se cumple por transitividad con cierto a no clave tal que $a \rightarrow b$ y $CC \rightarrow a$; y no por ser CC clave candidata).

Forma normal de Boyce-Codd (FNBC)

La definición de 3FN tiene deficiencias ya que no produce diseños satisfactorios cuando:

- Hay varias claves candidatas
- Las claves candidatas son compuestas
- Las claves candidatas se solapan.

FNBC es más restrictiva que 3FN. Decimos que una relación R está en FNBC $\Leftrightarrow \forall a \rightarrow b$ dependencia no trivial, se cumple que a es una superclave (a determina todos los demás atributos de la relación).
Equivale a decir que todo determinante (conjunto de atributos a en R tal que $a \rightarrow\rightarrow b$) es una clave candidata.

- Toda FNBC está en 3FN.
- Toda 3FN con **una única clave candidata** o **claves no solapadas** también está en FNBC.
- 3FN: siempre preservando dependencias.
- FNBC: siempre sin pérdidas, pero **no siempre preservando dependencias**.

ALGORITMO DE NORMALIZACIÓN DE FNBC

Input: una relación R y un conjunto asociado de dependencias Φ.
Output: un conjunto de relaciones que forman una descomposición sin pérdidas de R y que están en FNBC con respecto al conjunto de dependencias Φ.

1. Tablas = { R }
2. Si existe Φ no trivial con tal que no es superclave:
   2.1. Tablas=Tablas\{R}.
   2.2. Aplicar Teorema de Heath: dividir R en
        R1 con esquema
        R2 con esquema
   2.3. Tablas = Tablas FNBC(R1,Φ) // Llamada recursiva
   2.4. Tablas = Tablas FNBC(R2,Φ) // Llamada recursiva
3. Return (Tablas)

Nótese que es necesario conocer o calcular las claves candidatas de R, y por tanto en cada llamada recursiva pueden obtenerse distintos resultados válidos dependiendo del orden en que se consideren las dependencias. Una buena estrategia es empezar por las dependencias que rompan 2FN, seguir con las que rompan 3FN, y finalizar por las que rompan FNBC.
El resultado estará en FNBC con respecto al conjunto de dependencias inicial, pero no necesariamente con respecto a su cierre.

ALGORITMO DE CÁLCULO DE CLAVES CANDIDATAS

Dado el esquema R con un conjunto de dependencias funcionales Φ, el procedimiento para el cálculo de las claves candidatas es el siguiente:

1. Eliminación de atributos independientes: Construir, a partir de R, un conjunto de atributos R_{si} en el que se han eliminado los atributos independientes (son atributos primos, de ellos no se puede deducir ningún otro salvo ellos mismos).

2. Eliminación de atributos equivalentes: Construir, a partir de R_{si}, un conjunto de atributos R_{sie}en el que se han eliminado los atributos equivalentes, escogiendo uno de los dos atributos de cada equivalencia y sustituyendo el eliminado por el elegido en cada dependencia funcional de Φ en la que aparezca; Como resultado de este paso, puede darse el caso de que determinados atributos aparecen como independientes entre sí (de nuevo paso 1)
3. Se selecciona como primer candidato a clave candidata Kp cualquier determinante de R_{sie} que no sea determinado; Si no quedan más determinantes en R_{sie}que sean a la vez determinados, Kp es clave candidata y se pasa al paso 5. En caso contrario, se pasa al paso 4.
4. Selección de una clave de R_{sie} en el que pueden aparecer determinantes que puedan ser determinados:
    - Se construye el conjunto R'_{sie}eliminando atributos que aparecen en K_p^+, pero que ellos mismos no determinan a nadie (sólo aparecen a la dcha).
    - Se obtiene una clave provisional K'_p en R'_{sie}, con los determinantes de K_p y añadiendo a estos un nuevo determinante que sea determinado.
        - Si $K'_p = R'_{sie} \Rightarrow K'_p$, es una clave de R'_{sie}.
        - Si no, se añade un nuevo atributo que sea determinado y que no pertenezca al cierre de K'_p y se vuelve a comprobar.
    - Se repite la operación para cubrir todas las claves posibles.
    - Se añade a cada clave de R'_{sie}las obtenidas del paso 3 para obtener las claves de R_{sie}.
5. Añadir los atributos independientes a las claves obtenidas para R_{sie}.
6. Replicar las claves con las equivalencias eliminadas en el paso 2 para generar todas las claves.

ALGORITMO RECUBRIMIENTO MINIMAL DE UN CONJUNTO DE DEPENDENCIAS

Recubrimiento minimal de dependencias:
Se llama recubrimiento minimal o canónico de un conjunto de dependencias funcionales Φ y se nota por Φ' a un conjunto que cumpla:
$\Phi^+ = (\Phi')^+$ tal que cualquier dependencia que se puede obtener a través de Φ se puede obtener a través de Φ', y que cumpla que Φ′ está formada por dependencias con estructura mucho más <u>simple</u> que las de Φ (da igual donde haya más o menos, lo que importa es la simplicidad).

Proceso:

1. Obtención de $\Phi^{(1)}$ mediante aplicación de la regla de descomposición de Amstrong a todas las dependencias que tengan <u>parte derecha compuesta.</u>
2. Obtención de $\Phi^{(2)}$ mediante simplificación de la <u>parte izquierda</u> de las dependencias, eliminando atributos raros ($\alpha A \rightarrow B$ se dice que A es raro con respecto a $\alpha \Leftrightarrow A \in \alpha^+$, es decir, depende funcionalmente de α). Estos se suprimen.
3. Obtención de $\Phi^{(3)}$ eliminando dependencias redundantes (las que se puedan obtener a partir de las demás). Es más fácil comprobar si existe una relación entre el cierre de dependencias y el cierre de atributos: $a \rightarrow b \in (\Phi - \{a \rightarrow b\})^+ \Leftrightarrow b \in a^+_{\Phi-\{ab\}}$,se suprime.

DEPENDENCIAS MULTIVALUADAS Y CUARTA FORMA (4FN)

Sea una relación R con subconjuntos de atributos $X, Y \subset R$. Decimos que X multidetermina Y ($X \text{ ->> } Y$:

- X determina Y
- Y es independiente de $R - X - Y$

De forma menos intuitiva: para toda instancia r de R, dadas dos tuplas t,s de r que cumplen t[X]=s[X], existen dos tuplas u,v en r tales que:

1. u[X]=v[X]=t[X]=s[X]
2. u[Y]=t[Y] y además u[R-X-Y]=s[R-X-Y]
3. v[Y]=s[Y] y además v[R-X-Y]=t[R-X-Y]

- Si XY = R entonces X ->> Y se cumple trivialmente.
- X ->> Y sii X ->> R-X-Y

Una relación R está en 4FN si para toda dependencia multivaluada no trivial X ->> Y con Y no incluida en X se cumple que X es una superclave de R. Si R no está en 4FN debido a una dependencia multivaluada X ->> Y, la solución es dividir el esquema R en dos esquemas: XY por un lado y R-Y por otro, que formarán una descomposición sin pérdidas de R.

Hay más dependencias y formas normales. La 5FN se define en términos de Dependencias de Reunión: una relación R está en 5FN sii está en 4FN y todas las dependencias de reunión son implicadas por claves candidatas.